

Sistema Predictivo con MLOps para la Estimación de Carga de Trabajo y Producción en Cosechas Humano–Robot

Ivan García Fernández

Facultad de ingeniería

Universidad Andres Bello

Santiago, Chile.

i.garciafernandez@uandresbello.edu

Resumen—La creciente incorporación de robots colaborativos en entornos agrícolas genera desafíos inéditos para la planificación operativa de las cosechas. Este trabajo presenta el despliegue, bajo una arquitectura MLOps de nivel productivo, del pipeline predictivo de carga de trabajo humano (*workload*) y producción desarrollado para un escenario de cosecha de paltas humano–robot. El pipeline científico base —simulación multi-agente MATLAB con diseño factorial completo (144 corridas por escenario), ANOVA multifactorial tipo II como criterio de selección de variables y torneo de 12 familias de modelos de regresión con doble validación (CV 5-fold y hold-out)— alcanza R^2 de validación cruzada $\geq 0,98$ en las cuatro variables objetivo y muestra que la estrategia cooperativa aumenta la producción total en 34–64 % con incrementos de workload de solo 5–13 %. La contribución de este informe es la operacionalización de ese pipeline: una arquitectura de cinco capas que integra versionado de experimentos (MLflow sobre PostgreSQL) y de datos (DVC con doble remoto), servicio REST genérico multi-experimento (FastAPI tras nginx con TLS), autenticación (Keycloak/OAuth2 y API keys), observabilidad (Prometheus y Grafana), monitoreo de *drift* con reentrenamiento automático *champion/challenger* y CI/CD sobre *runner self-hosted*, todo contenedorizado con Docker. El sistema se valida mediante una pirámide de pruebas de seis niveles cuyas corridas *end-to-end* detectaron cuatro hallazgos reales —incluido un sesgo por sub-grupo que las métricas globales de R^2 enmascaran y que el lazo de reentrenamiento automático mitigó parcialmente sin intervención manual—, demostrando el valor de las prácticas de QA por segmento en el cruce entre agricultura de precisión e ingeniería de sistemas ML.

Index Terms—MLOps, machine learning, cosecha agrícola, interacción humano–robot, predicción de workload, ANOVA, FastAPI, MLflow, Docker, simulación multiagente

I. INTRODUCCIÓN Y ESTADO DEL ARTE

I-A. Contexto: agricultura, robótica e IA

La agricultura enfrenta presiones estructurales que impulsan la adopción acelerada de tecnologías de automatización. El envejecimiento de la mano de obra rural, la escasez estacional de trabajadores y la necesidad de maximizar la eficiencia operativa han convertido a los robots agrícolas en una solución con creciente viabilidad técnica y económica [6]. Revisiones sistemáticas recientes documentan una explosión en el uso de inteligencia artificial (IA) y robótica en agricultura a lo largo de la última década [11], [12], abarcando tareas que van desde

la detección de cultivos y la navegación autónoma [10] hasta la colaboración directa con operadores humanos [8].

En particular, los sistemas de cosecha semi-autónoma donde personas y robots comparten el espacio de trabajo representan un paradigma emergente con alto potencial productivo [9]. La interacción humano–robot (HRI) en contextos agrícolas introduce variables dinámicas que afectan directamente la productividad y la seguridad: el número de trabajadores, el ritmo de cosecha, la distribución espacial de la plantación, el tipo de actividad asignada y el nivel de utilización del robot [7]. La referencia base de este proyecto, Vásconez y Auat Cheein [1], evaluó producción y carga de trabajo en simulaciones multiagente de cosecha de paltas con estrategias colaborativas: sobre 41 corridas por escenario, la estrategia cooperativa aumentó la producción total en 23–85 % y la entrega en zona de carga en 15–80 %, con incrementos de workload de solo 1–16 %. Sin embargo, ese análisis quedó restringido a un conjunto finito de configuraciones simuladas y no propone modelos predictivos que permitan anticipar el comportamiento del sistema bajo configuraciones no simuladas antes de ejecutarlas en terreno.

Revisiones adicionales sobre tendencias en ML y robótica agrícola, sistemas de precisión y colaboración humano–robot confirman el creciente interés por integrar IA en todas las etapas del proceso productivo [20]–[24], [27]–[29].

I-B. Colaboración humano–robot, carga de trabajo y simulación

La literatura de HRC agrícola ha avanzado en tres frentes complementarios. Primero, la **interacción**: las revisiones sistemáticas identifican la colaboración física persona–robot en tareas de cosecha como el escenario de mayor potencial y, a la vez, de mayor complejidad [8], [9], con líneas activas en reconocimiento de movimiento y actividad humana para coordinar al robot [25], [26] y en verificación de seguridad ante el contacto físico [7]. Segundo, la dimensión **ergonómica**: la carga de trabajo física, estimable mediante tasas metabólicas normalizadas (ISO 8996 [4]), constituye un criterio de diseño tan relevante como la productividad al evaluar estrategias colaborativas [1]. Tercero, la evaluación por **simulación**: ante el

costo y la variabilidad de la experimentación en huertos reales, los entornos multiagente calibrados con parámetros de campo permiten generar datos controlados y comparar estrategias operativas de forma reproducible [1], [2]. Este proyecto se apoya en los tres frentes: adopta el simulador multiagente validado del estudio base, usa el workload metabólico como variable objetivo de primera clase, y añade sobre ellos una capa predictiva y de despliegue.

I-C. ML en agricultura y el problema de datos acotados

El aprendizaje automático ha demostrado ser una herramienta poderosa para la predicción de variables agrícolas a partir de datos operacionales [13], [14]. Estudios recientes muestran que métodos de *ensemble* como Random Forest y XGBoost ofrecen alto desempeño en tareas de regresión sobre datos tabulares en dominio agrícola [14], [15]. No obstante, un desafío frecuente en este tipo de proyectos es el tamaño acotado del conjunto de datos, ya que los experimentos provienen de simulaciones controladas con variaciones factoriales finitas. Trabajos en dominios relacionados han demostrado que estrategias como la validación k-fold y la regularización permiten obtener modelos robustos en escenarios de datos reducidos [19], lo que sustenta la viabilidad del enfoque adoptado.

I-D. La brecha MLOps en agricultura de precisión

A pesar del avance en modelos predictivos, existe una brecha significativa entre el desarrollo experimental de modelos ML y su despliegue como herramientas operativas accesibles y reproducibles. Esta brecha ha dado origen al campo de MLOps (*Machine Learning Operations*), que integra prácticas de ingeniería de software —integración continua, versionado de artefactos, contenedorización— con el ciclo de vida completo de los modelos ML [16]. Trabajos recientes demuestran que pipelines MLOps basados en MLflow y Docker permiten construir sistemas de predicción reproducibles, escalables y desplegables en entornos de producción [15], [17]. En agricultura de precisión, sin embargo, la adopción de MLOps es aún incipiente: Ozimek et al. [18] presentan uno de los pocos casos documentados de arquitectura ML desplegada con prácticas CI/CD en agricultura, lo que indica que este cruce representa tanto una necesidad urgente como una oportunidad de contribución concreta.

I-E. Propuesta

El presente proyecto se articula en dos frentes complementarios. El frente científico —desarrollado en el estudio *companion* de este trabajo [2]— cierra la brecha predictiva del trabajo base: extiende el simulador de [1] a un diseño factorial completo de 144 corridas por escenario, aplica ANOVA multifactorial tipo II como criterio fundamentado de selección de variables [3] y entrena 12 familias de modelos de regresión, alcanzando R^2 de validación cruzada $\geq 0,98$ en las cuatro variables objetivo y habilitando análisis de saturación, interpolación/extrapolación de dotaciones y escenarios multi-robot hipotéticos. El frente de ingeniería —objeto de este informe—

aborda la brecha remanente: convertir ese pipeline validado en un servicio operativo, seguro, observable y reproducible. Para ello se implementa una arquitectura MLOps *end-to-end* que automatiza el entrenamiento, registro, despliegue, monitoreo y reentrenamiento de los 8 modelos óptimos (2 escenarios $\times$ 4 *targets*), accesible vía API REST, apoyando decisiones de planificación más informadas, eficientes y reproducibles.

Las contribuciones concretas de este trabajo son: (1) una arquitectura MLOps de cinco capas, dirigida por configuración YAML, que automatiza el ciclo de vida completo de los 8 modelos del dominio HRI; (2) una API de inferencia genérica multi-experimento capaz de servir cualquier modelo del *registry* sin cambios de código; (3) una pirámide de QA de seis niveles con *gates* por sub-grupo (*slices*) que detecta sesgos invisibles para las métricas globales; (4) un esquema de versionado de datos con doble remoto DVC (local para CI/CD, en la nube para desarrollo); y (5) una validación *end-to-end* del stack desplegado, con hallazgos reales documentados y su evolución bajo reentrenamiento automático.

II. OBJETIVOS Y PREGUNTAS DE INVESTIGACIÓN

II-A. Objetivo General

Desarrollar e implementar un sistema predictivo basado en machine learning, desplegado bajo una arquitectura MLOps, capaz de estimar la carga de trabajo humano y la producción cosechada en escenarios de colaboración humano-robot, accesible como servicio REST en un entorno de testing similar a producción.

II-B. Objetivos Específicos

1. Consolidar los datos experimentales de simulación y el análisis estadístico del estudio base (ANOVA multifactorial tipo II [2], [3]), incorporando al pipeline reproducible del stack los tres conjuntos de variables derivados (Sets A, B y C) para las cuatro variables objetivo —*Total recolectado*, producción en zona de carga, *Total workload* y producción promedio humana— en ambos escenarios (8 *targets* en total).
2. Reproducir en el stack el torneo de modelos de regresión supervisada del estudio base, verificando que los ganadores por *slot* y sus métricas (R^2 , MAE, RMSE, sMAPE) sean consistentes con los reportados *offline* [2], e incorporando el Set C (términos Workers $\times$ Actividad) al entrenamiento de producción.
3. Implementar una arquitectura MLOps que integre versionado de experimentos (MLflow sobre PostgreSQL) y de datos (DVC), despliegue como API REST genérica (FastAPI), autenticación y TLS (Keycloak/OAuth2, *API key*), observabilidad (Prometheus + Grafana) y contenedorización (Docker), incorporando monitoreo de *drift* con reentrenamiento automático (*champion/challenger*) y pipelines de CI/CD que reemplacen la evaluación manual por un proceso automatizado y reproducible.
4. Validar el sistema en un entorno de testing basado en *docker-compose*, mediante una pirámide de pruebas de seis niveles (datos, modelo, API, *performance drift*,

model slices y contrato de API) que verifique disponibilidad del servicio, reproducibilidad de predicciones, ausencia de sesgo por sub-grupo y correctitud de los resultados.

II-C. Preguntas de Investigación

- **PI-1:** ¿Se reproducen, en el stack desplegado, las jerarquías de variables identificadas por el ANOVA tipo II del estudio base (dominancia de *Workers*, interacción *Workers*×*Actividad*), al contrastar el artefacto *feature_ranking.json* de los modelos en producción con la jerarquía estadística *offline*?
- **PI-2:** ¿El torneo automatizado del stack selecciona las mismas familias ganadoras que el estudio *offline* (regresión polinomial de grado 3 y *ensembles* de *boosting*), y qué aporta el Set C al desempeño y a la mitigación del sesgo por sub-grupo en producción?
- **PI-3:** ¿Qué métricas de calidad de servicio (latencia, reproducibilidad entre ejecuciones) alcanza el sistema desplegado en el entorno de testing con contenedores?

III. MARCO METODOLÓGICO

La metodología se organiza en tres bloques secuenciales, tal como se ilustra en la Figura 1. Los Bloques 1 y 2 corresponden a las Etapas 1–4 del diagrama (simulación, ANOVA, entrenamiento de modelos y *forecasting*): el pipeline científico, ya completado y reportado en el estudio base [2]. El Bloque 3 (Etapa 5, MLOps) operacionaliza ese pipeline en el camino de producción: automatiza el torneo de entrenamiento, el registro y el despliegue de los modelos, y sustituye la evaluación manual por monitoreo y reentrenamiento continuos, complementando —sin reemplazar— los *notebooks* de *forecasting* y *what-if* de la Etapa 4.

Un aspecto clave para la factibilidad es que tanto el pipeline científico como la infraestructura MLOps **ya se encuentran operativos y en una etapa madura**: los dos escenarios fueron consolidados en *simulation_all.csv* (284 observaciones válidas, versionadas con DVC), el análisis estadístico y el torneo de modelos están completos y en revisión editorial [2], y sobre este conjunto ya están entrenados, registrados y desplegados los 8 modelos de regresión descritos en la Sección III-C, junto con su servicio FastAPI genérico multi-experimento, backend PostgreSQL para MLflow, autenticación TLS/Keycloak/*API key*, observabilidad con Prometheus y Grafana, monitoreo de *drift* y pipelines de CI/CD operando sobre un *runner self-hosted*. La validación de este stack ya se ejecutó de punta a punta mediante la pirámide de pruebas de seis niveles, lo que arrojó hallazgos concretos aún pendientes de corrección (Sección IV). Las cinco semanas disponibles se concentran, por lo tanto, en: (i) incorporar al stack los conjuntos de variables B y C y el registro de *features* del estudio base, (ii) verificar la consistencia entre el torneo automatizado y la tabla comparativa final *offline*, y (iii) corregir los hallazgos de la validación *end-to-end* y re-certificar el stack bajo el dominio HRI. Esto hace el proyecto no solo factible,

sino que reduce el riesgo de ejecución a tareas de integración y corrección dirigida sobre un sistema ya en operación.

III-A. Bloque 1: Datos de Simulación y Escenarios

Los datos provienen del entorno de simulación multiagente MATLAB del estudio base [1], [2]: un huerto de paltos parametrizado a partir de una granja chilena, con 12 árboles dispuestos en 3 filas y una zona de carga en [15, 2] m. Cada corrida simula $T = 600$ s de faena con paso $\Delta t = 0,25$ s (2401 pasos discretos); una *crop unit* equivale a una palta cosechada y cada trabajador transporta hasta 15 unidades por viaje. El diseño experimental factorial completo varía cuatro factores: número de trabajadores (1, 3, 6, 8, 10, 12), fila de cultivo (1–3), modo de actividad (*Ground*, *Ladder*, *Picker*, *Mixed*) y posición inicial (fija/aleatoria), produciendo 144 corridas por escenario (288 en total). Las configuraciones con 8, 10 y 12 trabajadores son una extensión original respecto del simulador de [1], validado en terreno para dotaciones de 1, 3 y 6; el simulador reestructurado reproduce al original con desviaciones $\leq 0,5\%$ en workload y $\leq 4\%$ en producción promedio [2]. El conjunto consolidado desplegado en el stack (*simulation_all.csv*) contiene 284 observaciones válidas y 15 columnas (144 del Escenario A y 140 del B, tras descartar corridas incompletas).

III-A1. Escenario A — Cosecha Solo Humanos: Cada trabajador sigue una máquina de estados de seis fases: cosecha en árbol (hasta 15 unidades), descenso de escalera (5 s), traslado a zona de carga a $v = 0,75$ m/s, descarga (2 s), retorno al árbol y ascenso de escalera (5 s) (Figura 2). El workload físico se acumula en cada paso de tiempo según costos metabólicos por estado, conforme a ISO 8996:2004 [4]: $w_h(t) = w_h(t - \Delta t) + c_{\sigma(h,t)} \Delta t$, donde c_{σ} es el coeficiente metabólico (kcal/s) del estado activo del trabajador h ; el costo de cosecha c_{harv} difiere entre *ground*, *ladder* y *picker*, siendo el ascenso de escalera el estado más costoso.

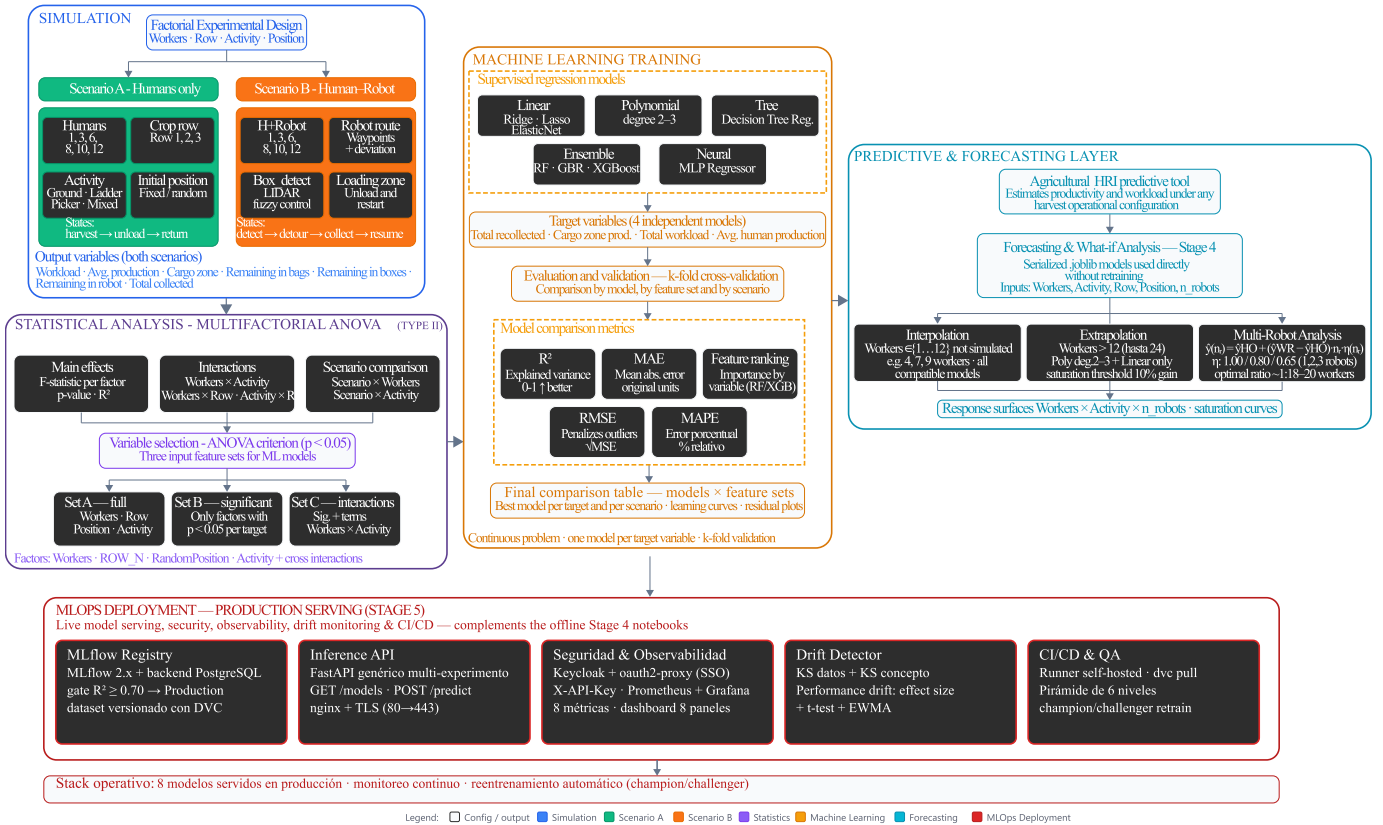


Figura 1. Arquitectura general del sistema en cinco etapas. Las Etapas 1–4 —simulación multiagente factorial, ANOVA multifactorial tipo II, torneo de 12 modelos de regresión y capa de *forecasting/what-if*— constituyen el pipeline científico del estudio base [2]. La Etapa 5 (*MLOps Deployment*, en rojo) es la contribución de este trabajo: automatiza el entrenamiento, registro y servicio en producción de los 8 modelos con seguridad, observabilidad, monitoreo de *drift* y CI/CD (detalle en la Figura 4), complementando los *notebooks* offline de la Etapa 4.

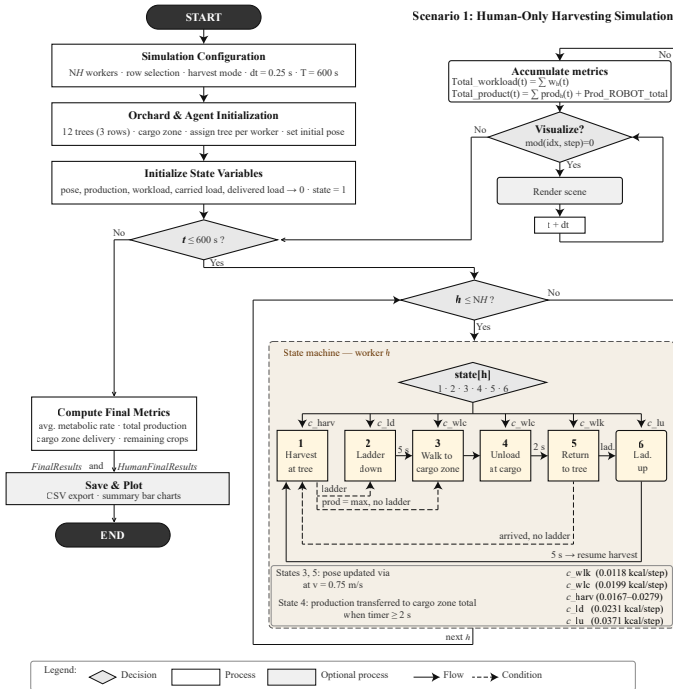


Figura 2. Escenario A: simulación de cosecha con agentes humanos únicamente. El trabajador h sigue una máquina de 6 estados que modela el ciclo completo cosecha—traslado—descarga, acumulando carga de trabajo metabólico (ISO 8996:2004 [4]) en cada paso de tiempo.

III-A2. Escenario B — Cosecha Humano-Robot: Extiende el Escenario A incorporando un robot autónomo de dirección en las cuatro ruedas (4WS) que asume la tarea de transporte. El robot percibe mediante un LIDAR de 13 haces ($r = 1,5 \text{ m}$, agrupados en 7 sectores angulares) y un detector de objetos (FoV = 120° , $r = 3 \text{ m}$); navega con tres controladores difusos Mamdani (fis_goal , fis_obstacle , fis_tracking) sobre rutas de *waypoints* generadas con PRM, inserta *waypoints* al detectar cajas nuevas, las recoge cuando la distancia es $< 0,75 \text{ m}$ y descarga al retornar a $< 0,5 \text{ m}$ de la zona de carga (Figura 3). El mecanismo clave de reducción de workload es estructural: los estados de traslado y descarga (4–5) se eliminan del ciclo humano —el trabajador deposita la caja junto al corredor del robot y retoma la cosecha de inmediato—, transfiriendo al robot los costos metabólicos de caminata con y sin carga [2].

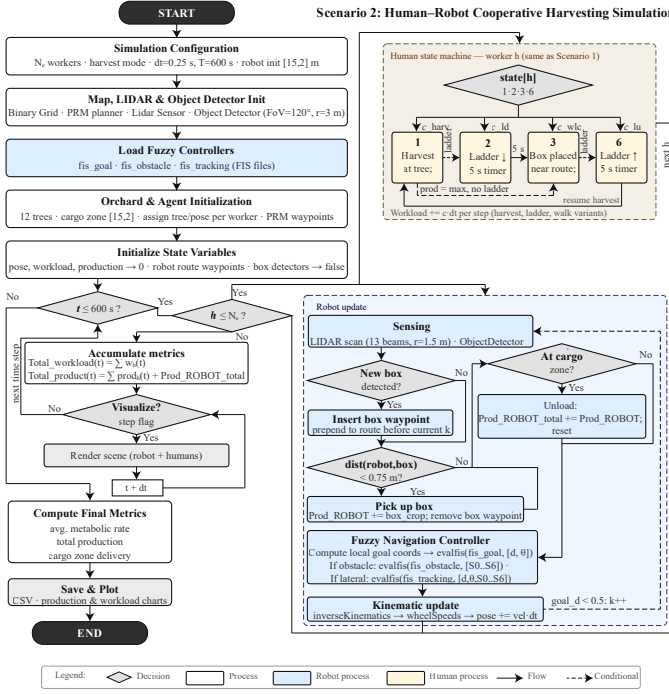


Figura 3. Escenario B: simulación de cosecha humano-robot. El robot actualiza su estado mediante LIDAR, detección de cajas, navegación difusa y cinemática inversa 4WS, acumulando producción robótica que se suma a la producción humana total.

Cada corrida produce ocho variables de salida. Cuatro constituyen los *targets* del modelado predictivo —*Total recolectado*, producción en zona de carga, *Total workload* y producción promedio humana—, modeladas de forma independiente por escenario (8 *slots* en total, Figura 1); fueron seleccionadas por su relevancia directa para la planificación operativa: las dos primeras caracterizan el rendimiento productivo de la faena, mientras que las dos últimas reflejan el esfuerzo y desempeño individual del equipo humano. Las tres variables residuales (*RemBags*, *RemBoxes*, *RemRobot*) se excluyen del modelado por ser componentes aditivas de *Total recolectado* —incluirlas introduciría fuga de información—, y *MetabolicRate* se conserva solo con fines descriptivos [2]. Las variables predictoras son *Humans*, *ROW_N*, *RandomPosition* y la actividad codificada *one-hot* (*Act_Ladder*, *Act_Mixed*, *Act_Picker*, con *ground* como categoría base). La preparación incluye eliminación de registros incompletos, tratamiento de atípicos y estandarización media-cero/varianza-uno dentro del pipeline de cada modelo.

III-B. Bloque 2: Análisis Estadístico y Entrenamiento de Modelos

III-B1. ANOVA Multifactorial (Tipo II) y Selección de Variables: Se aplica ANOVA multifactorial **tipo II** de forma independiente por escenario, con $\alpha = 0,05$; las sumas de cuadrados tipo II evalúan cada factor ajustando por los demás efectos principales sin penalizar por interacciones de orden superior, lo que las hace preferibles al tipo III cuando el modelo incluye

términos de interacción [3]. El modelo completo comprende los cuatro efectos principales (*Workers*, *CropRow*, *RandPos*, *Activity*) y cinco interacciones de dos vías (*Workers*×*Activity*, *Workers*×*CropRow*, *Activity*×*CropRow*, *Workers*×*RandPos*, *Activity*×*RandPos*), más un análisis combinado sobre el dataset agrupado para las interacciones *Escenario*×*Workers* y *Escenario*×*Activity*.

Con el criterio $p < 0,05$ se construyen tres conjuntos de entrada para el modelado: **Set A** (los cuatro factores, $p = 6$ *features* con la codificación *one-hot*), **Set B** (solo los factores significativos por *target*, con *fallback* al Set A si ninguno alcanzara significancia) y **Set C** (Set B más los tres términos de interacción *W*×*Ladder*, *W*×*Mixed* y *W*×*Picker*, $p = 9$). Antes de entrenar se verifica la multicolinealidad mediante factores de inflación de varianza (VIF), y las definiciones de los conjuntos se serializan en un registro JSON (*feature_registry.json*) que garantiza reproducibilidad entre entrenamiento y *forecasting*. La versión actualmente operativa del *pipeline* de producción (Sección III-C) entrena con el Set A; la incorporación de los Sets B y C al torneo del stack es parte del plan de trabajo (Semanas 1–2). Los resultados del análisis estadístico se reportan en la Sección IV.

III-B2. Torneo de Modelos de Regresión: Se entrenan y comparan 12 algoritmos para cada combinación escenario × variable objetivo × *feature set* (288 experimentos en el estudio base), abarcando cinco familias:

- Lineales: *LinearRegression*, *Ridge*, *Lasso*, *ElasticNet* ($\alpha = 0,1$)
- Polinomial de grados 2 y 3 (expansión polinomial + *Ridge*, $\alpha = 1,0$)
- Árbol de decisión (profundidad máx. 6)
- *Ensembles*: *Random Forest*, *Gradient Boosting* y *XGBoost* (200 estimadores) [14], [15]; *LightGBM*/*CatBoost* opcionales en el stack
- Red neuronal multicapa (MLP 64–32, ReLU, Adam) [19]

Cada modelo se evalúa con dos estrategias complementarias: validación cruzada 5-*fold* (media ± desviación de R^2 entre *folds*) y un *hold-out* independiente 80/20 con semilla fija, reportando R^2 , MAE, RMSE y MAPE/sMAPE más el *feature ranking* por modelo.

Este torneo está implementado en *model-trainer/train.py*, donde se entrena sobre los 8 *slots* independientes; el ganador de cada *slot* se registra en *MLflow* para su posterior promoción automática (Sección III-C).

III-B3. Etapa 4: Forecasting y Análisis Multi-Robot: Los modelos serializados (*.joblib*) se emplean sin reentrenamiento en tres modos: (i) interpolación de dotaciones no simuladas (p.ej. 4, 7 o 9 trabajadores), admisible para las 12 familias; (ii) extrapolación hasta 24 trabajadores empleando solo las familias polinomial y lineal —los métodos de árboles no extrapolan fuera de sus hojas—, con un umbral de saturación del 10% de ganancia para identificar cuándo el aporte del robot se vuelve operacionalmente marginal; y (iii) análisis multi-robot hipotético mediante un modelo de eficiencia de flota, $\hat{y}(n_r) = \hat{y}_{HO} + (\hat{y}_{WR} - \hat{y}_{HO}) n_r \eta(n_r)$,

con $\eta = 1,00/0,80/0,65$ para 1/2/3 robots [2], [5]. El stack MLOps (Bloque 3) operacionaliza la predicción puntual de esta etapa como servicio en línea, dejando los análisis *what-if* exploratorios en los *notebooks*.

III-C. Bloque 3: Arquitectura MLOps y Despliegue

Este bloque lleva el torneo del Bloque 2 al camino de producción: automatiza su ejecución, registro y despliegue mediante un *stack* MLOps de cinco capas, ilustrado en la Figura 4 (el detalle de la Etapa 5 de la Figura 1). A diferencia de un pipeline mínimo de registro y despliegue, esta arquitectura incorpora autenticación y observabilidad, monitoreo continuo de *drift* y reentrenamiento automático [16], alineándose con prácticas de MLOps de nivel productivo [15], [17], [18].

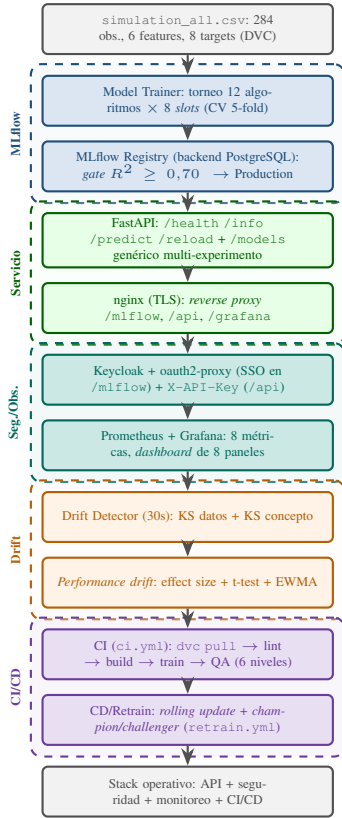


Figura 4. Pipeline MLOps de cinco capas (Bloque 3). Los 8 modelos del Bloque 2 se registran y versionan en MLflow sobre backend PostgreSQL (Capa 1), se exponen vía FastAPI genérico/nginx con TLS (Capa 2), quedan protegidos por SSO/API key e instrumentados con Prometheus/Grafana (Capa 3), se monitorean continuamente por *drift* con reentrenamiento *champion/challenger* (Capa 4) y se despliegan mediante CI/CD con *runner self-hosted* (Capa 5) [16]–[18].

III-C1. Capa 1 — Entrenamiento y Registro (MLflow):

El módulo *model-trainer* ejecuta el torneo de 12 algoritmos (Sección III-B) sobre 8 *slots* independientes (2 escenarios $\times$ 4 *targets*), cada uno validado con CV 5-fold. El modelo con mayor R^2 de cada *slot* se registra en el experimento MLflow *hri-harvesting* junto con un artefacto *feature_ranking.json*, y se promueve a *Production* si supera el *quality gate*

$R^2 \geq 0,70$ [15]. Los 8 modelos siguen la convención *hri*-{HumanOnly, WithRobot}-{TotalRecollected, CargoZoneProd, TotalWorkload, AvgProduction}, asegurando trazabilidad y versionado [17]. El backend de metadatos de MLflow migró de SQLite a PostgreSQL, habilitando escrituras concurrentes desde múltiples *runs*; el dataset *simulation_all.csv* se versiona con DVC bajo un esquema de *doble remoto*: un remoto local (*local-store*) usado exclusivamente por el *runner self-hosted* vía *dvc pull* en cada corrida de CI/CD —sin dependencias externas ni credenciales—, y un remoto secundario en Google Drive (autenticación OAuth) que permite a los desarrolladores sincronizar el dataset desde cualquier máquina sin depender de la ruta local de producción. Ambos remotos coexisten sin que el pipeline de CI/CD requiera modificación alguna.

III-C2. Capa 2 — Servicio de Inferencia (FastAPI + nginx): *inference-api* carga los 8 modelos *Production* en memoria al iniciar y expone cuatro *end-points* específicos de HRI: GET */health* (estado y número de modelos cargados), GET */info* (metadatos de escenarios, *targets* y *features*), POST */predict* (recibe escenario, número de trabajadores, fila de cultivo, posición y actividad, y retorna las cuatro predicciones en una sola llamada) y POST */reload* (recarga modelos sin reiniciar el contenedor). A estos se suma un conjunto **genérico multi-experimento**: GET */models* lista cualquier modelo en *Production* del *registry*, GET */models/{name}* devuelve su metadata y POST */models/{name}/predict* predice con cualquier modelo *sklearn* registrado, permitiendo servir nuevos experimentos (por ejemplo, un segundo dataset con distinto *MODEL_CONFIG*) sin cambios de código. Un *reverse proxy* nginx con TLS (certificado autofirmado, redirección 80→443) unifica el acceso bajo */mlflow/*, */api/* y */grafana/* [18].

III-C3. Capa 3 — Seguridad y Observabilidad: Dos mecanismos de autenticación conviven según el consumidor: */mlflow/* queda protegido por SSO vía Keycloak (realm *mlops*) y *oauth2-proxy*, que intercepta la petición en nginx mediante *auth_request* y redirige al login si la sesión no es válida; */predict* y */reload* en *inference-api*, en cambio, exigen un *header* X-API-Key, adecuado para consumo programático (*drift-detector*, CI/CD). */health* e */info* permanecen públicos. En observabilidad, *inference-api* expone en GET */metrics* cuatro métricas Prometheus (*inference_requests_total*, *inference_request_duration_seconds*, *inference_models_loaded*, *inference_predictions_total*) y *drift-detector* expone otras cuatro en el puerto 9091 (*drift_ks_pval*, *drift_detected_total*, *drift_consecutive_windows*, *drift_retrain_triggered_total*). Prometheus las recolecta cada 15 s y Grafana las visualiza en un *dashboard* de 8 paneles (modelos cargados, tasa de *requests*, latencia p50/p95/p99, *p-values* KS por *feature* y detecciones

de *drift*), accesible vía /grafana/.

III-C4. Capa 4 — Monitoreo y Reentrenamiento: El servicio `drift-detector` ejecuta cada 30 s, sobre una ventana de 30 muestras, tres pruebas inspiradas en [16]: (i) *data drift*, test de Kolmogorov–Smirnov por *feature* de entrada ($p < 0,05$); (ii) *concept drift*, KS sobre la distribución de predicciones en vivo versus referencia; y (iii) *performance drift*, una cascada de tres pruebas ($effect\ size > 0,05$, t -test con $p < 0,05$ y $|\Delta| > 0,02$, y EWMA con $\alpha = 0,3$) sobre R^2 , RMSE, MAE, sMAPE y *max error*. Tras dos ventanas consecutivas con *drift*, se dispara un ciclo *champion/challenger*: se entrena un *challenger* (Gradient Boosting) para los 8 *slots*, se compara su R^2 contra el *champion* vigente, se promueve si mejora y se invoca `POST /reload`. Todo se registra en los experimentos `MLflow drift-monitoring`, `performance-drift-monitoring` y `retraining-events`.

III-C5. Capa 5 — CI/CD y Pirámide de Testing: Un *runner self-hosted* (instalado vía `setup_runner.sh` como servicio `systemd`, con acceso directo al *socket* Docker) ejecuta tres flujos de GitHub Actions: `ci.yml` (en cada *push*/PR: `dvc pull`, `lint`, validación de `docker-compose`, `build`, entrenamiento y suite de QA completa), `cd.yml` (en *push* a `main`: `dvc pull`, *rolling update* de `inference-api` y luego `drift-detector+nginx`, con QA post-despliegue como *gate*) y `retrain.yml` (programado semanalmente, con *gates* configurables y `POST /reload` si aprueba) [18]. La suite de QA implementa una pirámide de **seis niveles** orquestada por `run_tests.py`, donde cada nivel bloquea al siguiente si falla: (1) *datos* — esquema, nulos, rangos y balance de escenarios; (2) *modelo* — $R^2 \geq 0,70$, $sMAPE < 20\%$, estabilidad CV y comparación contra *baseline*; (3) *API* — disponibilidad, esquema de respuesta y latencia < 500 ms; (4) *performance drift* — pruebas unitarias del detector ($effect\ size$, t -test, EWMA); (5) *model slices* — detección de sesgo por sub-grupo (actividad, rango de trabajadores, escenario), exigiendo $R^2 \geq 0,50$ por *slice* y una brecha de R^2 entre escenarios $\leq 0,25$; y (6) *API contract* — validación de esquema de respuesta para detectar cambios que rompan compatibilidad (*breaking changes*). Los resultados se registran en el experimento `MLflow quality-assurance`.

Los resultados de las corridas de validación de esta pirámide contra el stack desplegado, y los hallazgos que produjeron, se reportan en la Sección IV.

III-D. Plan de Trabajo

La Tabla I detalla la distribución de actividades en las cinco semanas disponibles.

Cuadro I
PLAN DE TRABAJO POR SEMANA

Sem.	Bloque	Actividades principales
1	B1–B2	Validación de <code>simulation_all.csv</code> (pirámide Niveles 1–2) e incorporación del registro de <i>features</i> del estudio base (<code>feature_registry.json</code> , Sets A/B/C) al <code>model-trainer</code>
2	B2	Extensión del torneo del stack a los Sets B y C (términos $W \times$ Actividad); verificación de consistencia de ganadores por <i>slot</i> contra la tabla comparativa <i>offline</i> [2]; registro en <code>MLflow</code>
3	B3	Corrección de los hallazgos (i) código 503 vs. 404 en <code>/models/{name}</code> , (ii) falsos positivos del detector de <i>performance drift</i> y (iv) conectividad del <code>test-runner</code> (URLs por nombre de servicio); nueva corrida de la pirámide
4	B3	Mitigación del hallazgo (iii), sesgo por <i>slice</i> en <code>WithRobot-AvgProduction</code> (reentrenamiento con Set C y/o ajuste de <i>gates</i> por segmento) y re-certificación <i>end-to-end</i> del stack
5	B3	Pruebas de integración finales (pirámide de 6 niveles); documentación y entrega

Cuadro II
STACK TECNOLÓGICO

Componente	Herramienta
Lenguaje	Python 3.10+
Datos de entrada	<code>simulation_all.csv</code> (284 obs., 8 <i>targets</i>)
ML	<code>scikit-learn</code> , <code>XGBoost</code> , <code>LightGBM</code> , <code>CatBoost</code>
Seguimiento & registro	<code>MLflow 2.x</code> (Tracking + Registry, backend <code>PostgreSQL</code>)
Versionado de datos	<code>DVC</code> (remoto local + Google Drive)
API REST	<code>FastAPI</code> + <code>Uvicorn</code> (genérica multi-experimento)
Reverse proxy	<code>nginx</code> (TLS)
Autenticación	<code>Keycloak</code> + <code>oauth2-proxy</code> (SSO), <i>API key</i>
Observabilidad	<code>Prometheus</code> + <code>Grafana</code>
Contenedorización	<code>Docker</code> + <code>docker-compose</code>
CI/CD	GitHub Actions (<i>runner self-hosted</i>)
Monitoreo de <i>drift</i>	<code>scipy</code> (KS test), <code>statsmodels</code> (EWMA)
Testing	<code>pytest</code> (6 niveles)
Análisis estadístico	<code>pandas</code> , <code>scipy</code> , <code>statsmodels</code>
Control de versiones	<code>Git</code> / <code>GitHub</code>

IV. RESULTADOS

IV-A. Pipeline Científico (Etapas 1–4)

El ANOVA tipo II identificó los cuatro factores de entrada como estadísticamente significativos ($p < 0,001$) para los cuatro *targets* en ambos escenarios, con R^2 del modelo $\geq 0,95$. *Workers* dominó la jerarquía con el mayor estadístico

F en todas las respuestas; *RandomPosition* fue el predictor más débil (no significativo para workload y producción promedio en el escenario humano-robot, donde la estrategia dinámica de *waypoints* absorbe parte de la variabilidad posicional); y la interacción *Workers*×*Activity* resultó significativa ($p < 0,001$) para los *targets* de productividad. Dos consecuencias directas para el modelado: el Set B resultó idéntico al Set A (todos los factores son necesarios) y la interacción detectada justifica los términos del Set C. El chequeo VIF ($\leq 2,0$) confirmó la ortogonalidad del diseño factorial [2].

En el torneo de 288 experimentos, la Tabla III resume los ganadores por *slot*: la regresión polinomial de grado 3 domina los *targets* de productividad en el escenario solo-humanos —consistente con rendimientos decrecientes suaves al crecer la dotación—, mientras que los *ensembles* de *boosting* dominan el escenario humano-robot, cuyo *target* más difícil (zona de carga, R^2 CV = 0,981) refleja la variabilidad estocástica de la inserción dinámica de *waypoints*. Ninguna familia cae bajo R^2 CV = 0,92 salvo el MLP (hasta 0,84 en producción promedio), penalizado por el tamaño muestral ($n \leq 144$); la concordancia entre CV y *hold-out* descarta sobreajuste. El *feature ranking* por Random Forest corrobora la jerarquía ANOVA (*Workers* con importancia $\approx 0,85$ – $0,88$ en producción total y workload) e identifica el término *W*×*Ladder* como predictor dominante de la producción promedio en humano-robot —un hallazgo recuperable solo mediante el Set C [2].

Cuadro III
MEJOR MODELO POR *slot* EN EL ESTUDIO BASE (CV 5-fold) [2]

Escenario	Target	Modelo (set)	R^2 CV
Solo humanos	Total recolectado	Polinomial g.3 (B)	0,999
Solo humanos	Zona de carga	Polinomial g.3 (B)	0,998
Solo humanos	Workload	Grad. Boosting (C)	0,9997
Solo humanos	Prod. promedio	XGBoost (C)	0,992
Humano-robot	Total recolectado	Grad. Boosting (A)	0,9998
Humano-robot	Zona de carga	XGBoost (C)	0,981
Humano-robot	Workload	Grad. Boosting (A)	1,000 [†]
Humano-robot	Prod. promedio	Árbol decisión (A)	1,000 [†]

[†]Ajuste perfecto atribuible al determinismo del simulador (memorización): constituye una cota superior, sin evidencia de generalización a datos de terreno [2].

A nivel de sistema, la estrategia cooperativa con un robot aumenta la producción total recolectada en 34–64 % (máximo en *ground*, +64 %) y la entrega en zona de carga en 16–41 %, con un incremento de workload humano de solo 5–13 %, extendiendo los rangos reportados por [1] a un espacio de diseño más amplio y sistemático. En la Etapa 4, no se detecta saturación del aporte del robot dentro del rango entrenado (umbral del 10 %), pero la producción en zona de carga exhibe un máximo en ≈ 18 – 20 trabajadores —el transporte robótico se vuelve la restricción activa—, lo que sugiere un ratio robot:trabajadores de $\approx 1:18$ – 20 como conjetura estructurada de diseño; a dotación 12, el despliegue de 2 y 3 robots proyecta ganancias medias de +74 % y +90 % respectivamente [2].

IV-B. Despliegue y Operación del Stack

El stack completo opera sobre Docker Compose con 11 servicios (9 de larga vida y 2 *one-shot*: *model-trainer* y *test-runner*), con los 8 modelos en *Production* cargados en memoria por la API de inferencia. Las verificaciones operativas confirmaron el comportamiento esperado de punta a punta: una predicción de los cuatro *targets* en una sola llamada autenticada (*X-API-Key*) vía *nginx* con TLS retornó valores coherentes con la simulación (producción promedio ≈ 82 unidades para 6 trabajadores en solo-humanos, frente a ≈ 80 del estudio base) dentro del *gate* de latencia de 500 ms; el SSO Keycloak/oauth2-proxy protege */mlflow/* (redirección 302 verificada); Grafana sirve el *dashboard* de 8 paneles; y Prometheus mantiene sus dos *targets* de *scrape* en estado up. El esquema de doble remoto DVC se verificó de punta a punta: eliminado el dataset local (archivo y caché), *dvc pull* desde el remoto de Google Drive lo restauró íntegro con hash MD5 idéntico. Durante una semana de operación continua, el lazo de la Capa 4 ejecutó múltiples ciclos de reentrenamiento automático, acumulando cientos de versiones por *slot* en el *registry* (416–690 en los *slots* inspeccionados) sin intervención manual.

IV-C. Validación End-to-End: Pirámide de Seis Niveles

La Tabla IV resume la corrida completa más reciente de la pirámide contra el stack desplegado.

Cuadro IV
RESULTADOS DE LA PIRÁMIDE DE SEIS NIVELES (CORRIDA COMPLETA CONTRA EL STACK DESPLEGADO)

Nivel	Resultado	Detalle
1 — Datos	✓ PASS	28/28
2 — Modelo	✓ PASS	82/82
3 — API	× FAIL	36/37 — hallazgo (i)
4 — <i>Perf. drift</i>	× FAIL	16/19 — hallazgo (ii)
5 — <i>Model slices</i>	× FAIL	60/61 — hallazgo (iii)
6 — Contrato API	✓ PASS	26/26

La primera corrida *end-to-end* arrojó tres hallazgos concretos, pendientes de corrección antes de la re-certificación: (i) *GET /models/{name}* devuelve código HTTP 503 en vez de 404 para modelos inexistentes; (ii) el detector de *performance drift* produce falsos positivos (RMSE/MAE) en sus propias pruebas unitarias sobre datos estables; y (iii) el modelo *WithRobot-AvgProduction* exhibía sesgo real por sub-grupo en las actividades *harv_ground* ($R^2 = 0,21$) y *harv_ladder* ($R^2 = 0,00$), ambas por debajo del *gate* de Nivel 5. El tercer hallazgo es particularmente ilustrativo: para ese mismo *target*, el estudio base reporta un R^2 global de validación cercano a 1,0 —cota superior atribuible al determinismo del simulador [2]—, de modo que solo la evaluación por sub-grupos revela la degradación por segmento que la métrica global enmascara.

La segunda corrida (Tabla IV), ejecutada tras una semana de ciclos de reentrenamiento automático, mostró que el sesgo se redujo sin intervención manual: el *slice harv_ground* superó

el *gate* con un *champion* más reciente promovido por el lazo *champion/challenger*, persistiendo únicamente *harv_ladder* ($R^2 = 0,00$, $n = 36$). La misma corrida reveló un cuarto hallazgo, de infraestructura: el *entrypoint* del *test-runner* reescribe las URLs internas de servicio hacia la IP del *gateway* de Docker, un patrón que quedó inoperante al restringirse los puertos publicados a 127.0.0.1 durante el endurecimiento de seguridad, afectando también al paso de QA en CI (se validó un *workaround* con IPs internas de contenedor).

V. DISCUSIÓN

Métricas globales vs. evaluación por sub-grupos. El contraste entre el R^2 global $\approx 1,0$ y el $R^2 = 0,00$ del *slice harv_ladder* para el mismo modelo demuestra que los *gates* agregados son insuficientes como criterio de promoción a producción. El Nivel 5 de la pirámide convierte el sesgo por segmento en un *gate* bloqueante y, en este proyecto, detectó un defecto real que ninguna métrica global habría reportado — el argumento central a favor de integrar la evaluación por *slices* en el ciclo de CI/CD y no solo en el análisis exploratorio.

Alcance y límites del reentrenamiento automático. El lazo *champion/challenger* corrigió el *slice harv_ground* sin intervención humana, evidencia de que la Capa 4 aporta mantenimiento real del desempeño. Sin embargo, *harv_ladder* persistió a través de múltiples ciclos, lo que sugiere una limitación estructural y no un problema de actualización. La explicación más plausible conecta directamente con el análisis estadístico: el estudio base identificó el término de interacción $W \times Ladder$ como el predictor dominante de la producción promedio en el escenario humano-robot [2], y ese término no existe en el Set A con el que hoy entrena el *pipeline* de producción. La hipótesis de mitigación más directa del hallazgo (iii) es, por lo tanto, incorporar el Set C al torneo del stack (Semanas 1–2), cerrando el círculo entre el análisis *offline* y el comportamiento del modelo desplegado.

MLOps en agricultura de precisión. La arquitectura demuestra que prácticas de nivel productivo —SSO, TLS, observabilidad, CI/CD sobre *runner self-hosted*, versionado dual de datos— son viables en un dominio agrícola con recursos modestos (un solo host), en línea con los escasos casos documentados [15]–[18]. El hallazgo (iv) ilustra, además, el costo de integración que el endurecimiento de seguridad impone sobre la propia infraestructura de pruebas: exactamente el tipo de fricción operativa que separa un prototipo académico de un sistema operable, y que rara vez se reporta en la literatura.

Limitaciones. Primero, la generalización está acotada por el simulador: los $R^2 = 1,000$ observados reflejan memorización de un sistema determinista y los modelos no deben usarse en operaciones reales sin validación de campo [2]. Segundo, el entorno de despliegue es de *testing* (certificado autofirmado, host único, secretos locales), no un ambiente productivo con gestión centralizada de secretos y alta disponibilidad. Tercero, los factores de eficiencia multi-robot $\eta(n_r)$ para $n_r > 1$ son supuestos teóricos [5], no calibrados por simulación. Cuarto, el tamaño muestral ($n \leq 144$ por escenario; $n = 36$ por *slice*

de actividad) limita la potencia estadística de los *gates* por sub-grupo.

VI. CONCLUSIONES Y TRABAJO FUTURO

Se diseñó, desplegó y validó de punta a punta un sistema MLOps de cinco capas que operacionaliza el pipeline predictivo HRI del estudio base: 8 modelos de regresión servidos tras nginx/TLS con doble mecanismo de autenticación, observabilidad con Prometheus/Grafana, monitoreo de *drift* con reentrenamiento automático *champion/challenger*, versionado de datos con doble remoto DVC y CI/CD reproducible custodiado por una pirámide de QA de seis niveles.

Respecto de las preguntas de investigación: **PI-1** — la jerarquía del ANOVA tipo II (dominancia de *Workers*, interacción $W \times Activity$) se corrobora en los *rankings* de importancia del estudio base; su verificación sistemática contra el artefacto *feature_ranking.json* de los modelos en producción queda programada junto con la integración de los Sets B/C. **PI-2** — *offline* dominan la regresión polinomial de grado 3 y los *ensembles* de *boosting*; en el stack, los ciclos *champion/challenger* (con *challenger* Gradient Boosting) han promovido repetidamente nuevas versiones, y la comparación formal por *slot* con los tres *feature sets* es parte del plan (Semanas 2–3). **PI-3** — el servicio cumple el *gate* de latencia (< 500 ms) bajo TLS y autenticación, mantiene estable su contrato de API (26/26) y la reproducibilidad de las predicciones queda garantizada por el versionado de modelos del *registry*.

El trabajo futuro inmediato es la agenda que la propia pirámide produjo: corregir los hallazgos (i), (ii) y (iv), recertificar el stack, e incorporar el Set C al entrenamiento de producción como mitigación del hallazgo (iii). En el mediano plazo: validación de campo de los modelos —la frontera que el determinismo del simulador impone—, simulación multi-robot explícita que sustituya los η hipotéticos, y extensión multi-experimento del stack con un segundo dataset real aprovechando la API genérica y el diseño dirigido por configuración [2].

REFERENCIAS

- [1] J. P. Vázquez y F. A. Auat Cheein, “Workload and production assessment in the avocado harvesting process using human-robot collaborative strategies,” *Biosystems Engineering*, vol. 223, pp. 56–77, 2022. DOI: 10.1016/j.biosystemseng.2022.08.010.
- [2] I. García Fernández y J. P. Vázquez, “Predicting workload and production in a human-robot collaborative avocado harvesting scenario: a machine learning approach,” manuscrito en revisión, *Biosystems Engineering*, 2026.
- [3] Ø. Langsrud, “ANOVA for unbalanced data: use Type II instead of Type III sums of squares,” *Statistics and Computing*, vol. 13, pp. 163–167, 2003. DOI: 10.1023/A:1023260610025.
- [4] ISO 8996:2004, “Ergonomics of the thermal environment — Determination of metabolic rate,” International Organization for Standardization, Ginebra, 2004.
- [5] L. Emmi, M. González-de-Soto, G. Pajares y P. González-de-Santos, “New trends in robotics for agriculture: integration and assessment of a real fleet of robots,” *The Scientific World Journal*, vol. 2014, art. 404059, 2014. DOI: 10.1155/2014/404059.
- [6] A. Willekens *et al.*, “Development of an agricultural robot task-map operation framework,” *Journal of Field Robotics*, 2025. DOI: 10.1002/rob.70003.

- [7] L. Guevara *et al.*, "Probabilistic model-checking of collaborative robots: a human injury assessment in agricultural applications," *Computers and Electronics in Agriculture*, vol. 218, p. 108987, 2024. DOI: 10.1016/j.compag.2024.108987.
- [8] G. Adamides y Y. Edan, "Human-robot collaboration systems in agricultural tasks: a review and roadmap," *Computers and Electronics in Agriculture*, vol. 204, p. 107541, 2023. DOI: 10.1016/j.compag.2022.107541.
- [9] L. Benos *et al.*, "Human-robot interaction in agriculture: a systematic review," *Sensors*, vol. 23, no. 15, p. 6776, 2023. DOI: 10.3390/s23156776.
- [10] M. Wakchaure, B. K. Patle y A. K. Mahindrakar, "Application of AI techniques and robotics in agriculture: a review," *Artificial Intelligence in the Life Sciences*, vol. 3, p. 100057, 2023. DOI: 10.1016/j.aillsi.2023.100057.
- [11] A. Hamrani *et al.*, "AI and robotics in agriculture: a systematic and quantitative review of research trends (2015–2025)," *Crops*, vol. 5, no. 5, p. 75, 2025. DOI: 10.3390/crops5050075.
- [12] N. Aijaz *et al.*, "Artificial intelligence in agriculture: advancing crop productivity and sustainability," *Journal of Agriculture and Food Research*, p. 101762, 2025. DOI: 10.1016/j.jafr.2025.101762.
- [13] Y. N. Kuan, K. M. Goh y L. L. Lim, "Systematic review on machine learning and computer vision in precision agriculture: applications, trends, and emerging techniques," *Engineering Applications of Artificial Intelligence*, p. 110401, 2025. DOI: 10.1016/j.engappai.2025.110401.
- [14] P. Sharma *et al.*, "Predicting agriculture yields based on machine learning using regression and deep learning," *IEEE Access*, vol. 11, pp. 101416–101430, 2023. DOI: 10.1109/access.2023.3321861.
- [15] K. E. Arunkumar *et al.*, "Predicting dry matter intake in cattle at scale using gradient boosting regression techniques and MLflow containerization," *Journal of Animal Science*, 2025. DOI: 10.1093/jas/skaf041.
- [16] IEEE Computer Soc., "AutoDrift: a forecast-aware concept drift detection and retraining pipeline in MLOps with CMAPSS," *Proc. 11th IEEE Int. Conf. Big Data Computing Service and Machine Learning Applications (BigDataService)*, Tucson, AZ, 2025. DOI: 10.1109/bigdata-service65758.2025.00019.
- [17] R. Singh y B. Roy, "Scalable earthquake magnitude prediction using spatio-temporal data and model versioning," *Scientific Reports*, 2025. DOI: 10.1038/s41598-025-00804-x.
- [18] J. Ozimek *et al.*, "AI-driven machine learning architecture for scalable irrigation detection in precision agriculture: a case study with CropX," *Proc. ECSA 2025 (Software Architecture)*, 2026. DOI: 10.1007/978-3-032-02138-0_27.
- [19] V. Barkov *et al.*, "Modern neural networks for small tabular datasets: the new default for field-scale digital soil mapping?," *European Journal of Soil Science*, 2025. DOI: 10.1111/ejss.70299.
- [20] K. Halder *et al.*, "High-resolution maize yield mapping across Africa using earth observation and machine learning," *Science of Remote Sensing*, p. 100344, 2026. DOI: 10.1016/j.srs.2025.100344.
- [21] M. F. Taha *et al.*, "Emerging technologies for precision crop management towards Agriculture 5.0: a comprehensive overview," *Agriculture*, vol. 15, no. 6, p. 582, 2025. DOI: 10.3390/agriculture15060582.
- [22] M. Li *et al.*, "CNN-MLP-based configurable robotic arm for smart agriculture," *Agriculture*, vol. 14, no. 9, p. 1624, 2024. DOI: 10.3390/agriculture14091624.
- [23] A. Thakur *et al.*, "Physical artificial intelligence for powering the next revolution in robotics," *Journal of Computing and Information Science in Engineering*, 2025. DOI: 10.1115/1.4070122.
- [24] G. Mohyuddin *et al.*, "Evaluation of machine learning approaches for precision farming in smart agriculture system: a comprehensive review," *IEEE Access*, vol. 12, pp. 56432–56452, 2024. DOI: 10.1109/access.2024.3390581.
- [25] V. Moysiadis *et al.*, "Human-robot interaction through dynamic movement recognition for agricultural environments," *AgriEngineering*, vol. 6, no. 3, 2024. DOI: 10.3390/agriengineering6030146.
- [26] L. Benos *et al.*, "Explainable AI-enhanced human activity recognition for human-robot collaboration in agriculture," *Applied Sciences*, vol. 15, no. 2, p. 650, 2025. DOI: 10.3390/app15020650.
- [27] IEEE, "A review on the use of AI and robotics in agricultural practices," *Proc. 2nd Int. Conf. Intelligent Cyber Physical Systems and IoT (ICOICI 2024)*, 2024. DOI: 10.1109/icoici62503.2024.10696164.
- [28] J. Logeshwaran *et al.*, "Improving crop production using an agro-deep learning framework in precision agriculture," *BMC Bioinformatics*, vol. 25, p. 319, 2024. DOI: 10.1186/s12859-024-05970-9.
- [29] P. Zhang *et al.*, "Pixel-scene-pixel-object sample transferring: a labor-free approach for high-resolution plastic greenhouse mapping," *IEEE*

Transactions on Geoscience and Remote Sensing, vol. 61, 2023. DOI: 10.1109/tgrs.2023.3257293.